

Trabajo Práctico Integrador Programación I
Sistema de gestión de Países con manejo de archivos y codificación en Python

Alumnos:
Claudio Legrand, Christian Herrero
Grupo 9

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional
Programación 1

Profesores:
Ariel Enferrel, Martín A. García, Cinthia Rigoni

Tutor Comisión 8:

Luciano Chirolí

Tutor Comisión 19:

Flor Camila Gubiotti

08 de junio de 2026

TABLA DE CONTENIDO

Marco Teórico	3
Estructura de datos	3
Diccionarios	3
Modularización	4
Archivos CSV	5
Metodología utilizadas	5
Objetivos del proyecto	5
Decisiones técnica y arquitectura	6
Boceto	6
Diagrama de flujo	7
Ejemplo de código	8
dificultades y conclusiones	9
Bibliografía	10
Link Repositorio y video	10

Marco Teórico

Presentamos los conceptos y estructuras utilizadas en nuestro programa explicando su funcionalidades de cada uno.

ESTRUCTURAS DE DATOS:

Las estructuras de datos nos permiten almacenar Información para organizarlos y manipularlos de manera eficiente.

Uno de estos conceptos esenciales es el array, una estructura que puede contener múltiples valores, lo que facilita la escritura de programas que manejan grandes cantidades de datos. Y dado que las listas en sí mismas pueden contener otras listas, puedes usarlas para organizar los datos en estructuras jerárquicas.

Forma de acceder a los datos de una estructura:

Cada elemento es accesible mediante posiciones llamadas **índices**, que siempre comienzan desde cero.

El índice es el número que representa la posición de cada elemento dentro del array.

Posición del elemento	1	2	3	4	5
Valor	Lunes	Martes	Miércoles	Jueves	Viernes
Índice	0	1	2	3	4

Creación propia: Excalidraw

En nuestro programa utilizamos como estructura de datos una lista con diccionarios dentro de ella, armando así una **lista de diccionarios**.

LOS DICCIONARIOS:

Proporciona una forma flexible de acceso y organización de datos, un diccionario es una colección mutable de muchos valores. A diferencia de las listas, los diccionarios tienen un índice llamado claves, y una clave con su valor asociado se llama par de clave-valor.

Estructura de un diccionario:

```
diccionario = { 'clave' : 'valor' }
```

Creación propia: Excalidraw

La estructura de datos de nuestro programa es una lista de diccionario la cual utilizamos para poder almacenar aquellos datos de los países que el usuario quiere ingresar.

MODULARIZACIÓN

Modularizados nuestro programa a través de funciones ya que nos permite tener un código más limpio y más optimizado para la codificación y la gestión de errores.

La modularización va a estar dividida (o las funciones) a través de las funcionalidades principales como por ejemplo agregar países o filtrarlos.

Funciones:

Una función es como un proceso dentro de un proceso, también se lo considera como una secuencia de sentencias que realiza una computación.

Cuando definimos una función le especificamos un nombre, un argumento y la secuencias de sentencias, donde después, podemos llamar esa función por su nombre y reutilizar ese algoritmo, .

Un argumento es el dato de entrada que va a ser procesado ya estando dentro de la función se lo asigna a una variable llamadas parámetros

Estructura de una función:

```
def nombre_de_funcion(argumento):  
  
    algoritmo
```

ARCHIVOS CSV:

En nuestro programa cuando está en ejecución, los datos viven en la RAM por lo tanto, cuando no está en ejecución perdemos esos datos, para poder guardarlos de forma permanente y reutilizarlos en otras ejecuciones, deberíamos almacenarlos en archivos.

El archivo que utilizamos en nuestro programa es de formato **CSV** que son similares a planillas donde cada línea representa una fila, y los valores están separados por coma.

Las operaciones o funcionalidades que podemos realizar en Python con los archivos son de abrir, leer, escribir y cerrar desde nuestro programa.

METODOLOGÍA UTILIZADA:

- **Método Burbuja:** Para ordenar los países se utilizó el algoritmo de ordenamiento de burbuja mediante dos bucles for anidados. El bucle externo controla las pasadas sobre la lista, restando en cada iteración los elementos que ya quedaron ordenados al final para evitar comparaciones innecesarias, representa ese elemento que ya está ordenado. El bucle interno compara directamente el elemento actual con el siguiente (+ 1) e intercambian sus posiciones en el arreglo si se cumple la condición de ordenamiento.

Toda esta información fue extraída del material de la cátedra brindado en la materia de programación 1, véase en Referencias Bibliográficas.

OBJETIVO DEL PROYECTO:

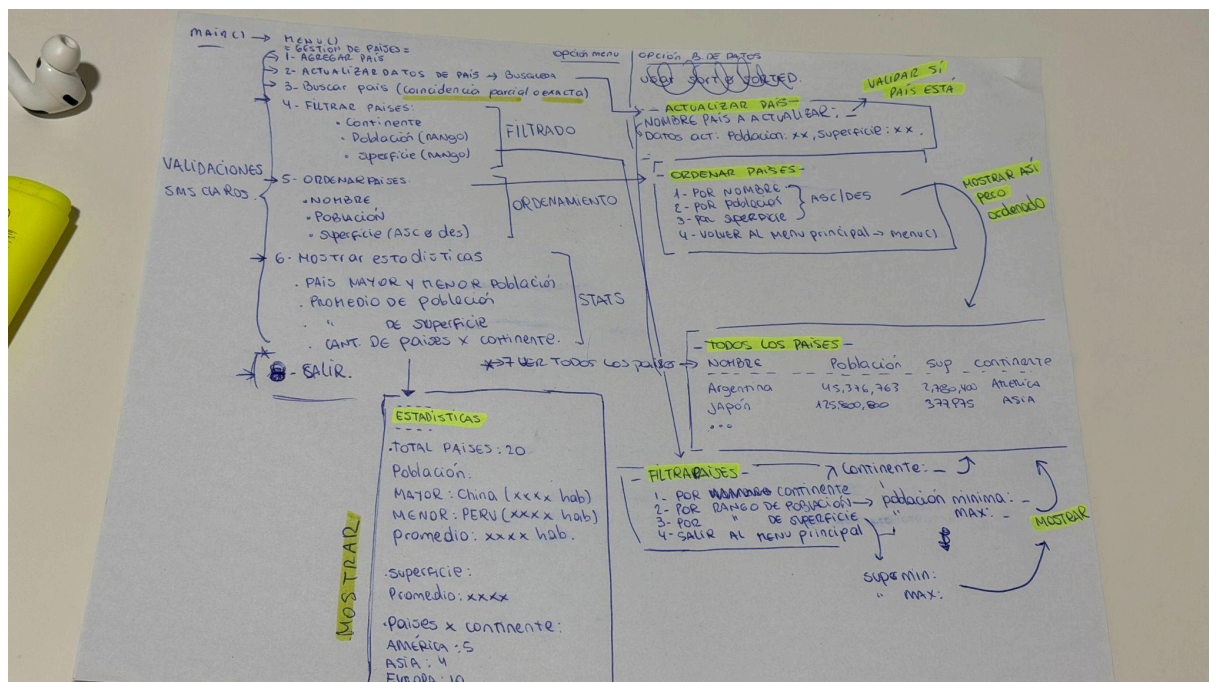
Desarrollar una aplicación en Python que permita gestionar información sobre países, aplicando listas, diccionarios, funciones, estructuras condicionales y repetitivas, ordenamientos y estadísticas. El sistema debe ser capaz de leer datos desde un archivo CSV, realizar consultas y generar indicadores clave a partir del dataset. El objetivo principal es afianzar el uso de estructuras de datos, modularización con funciones y técnicas de filtrado/ordenamiento, aplicando los conceptos aprendidos en Programación 1.

DECISIONES TÉCNICAS Y ARQUITECTURA:

Al empezar a diseñar el programa, antes de codificar, nos reunimos para establecer los puntos críticos de nuestro proyecto y diagramar el flujo que debía tener nuestro sistema.

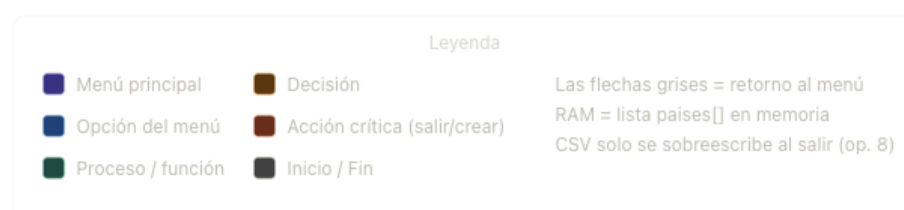
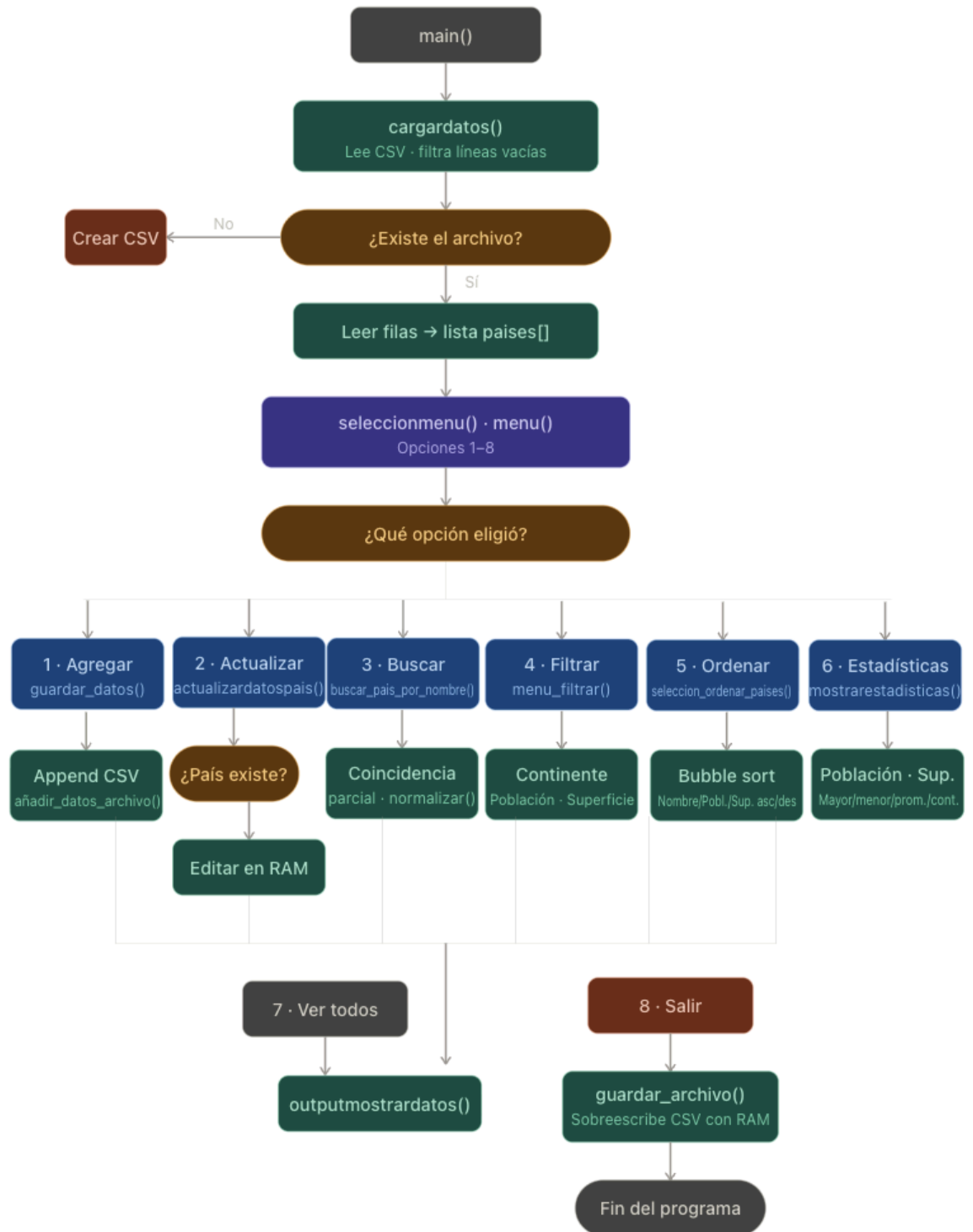
Comenzando con un boceto y presentación de las ideas que teníamos, a través de menús, submenús e ideas que nos iban surgiendo en el camino; al ver el sistema de forma más visual, pudimos ir a codificar con mejor entendimiento de nuestros pasos a realizar y sabiendo que hacía cada parte del código.

Boceto de pensamiento principal:



Boceto realizado por los alumnos

Diagrama de flujo del sistema:



Ejemplos de código:

Probamos el programa con la funcionalidad de **agregar un país** con sus datos y guardarlo en el archivo CSV

```
Seleccione una opción (1-8): 1
Ingrese el nombre del país para agregar [Use 'salir' para volver al menu principal]: Colombia
Ingrese la población del país: 53057212
Ingrese la superficie del país: 1141748
Seleccione continente
  1 - America
  2 - Europa
  3 - Oceania
  4 - Africa
  5 - Asia

Seleccione continente: 1
País agregado y guardado correctamente!
```

Al final se guardó correctamente, para poder comprobarlo lo verificamos viendo en el inventario:

```
Seleccione una opción (1-8): 7
```

Nombre	Población	Superficie	Continente
Argentina	45,376,763	2,780,400	América
Japón	125,800,000	377,975	Asia
Brasil	213,993,437	8,515,767	América
Alemania	83,149,300	357,022	Europa
Colombia	53,057,212	1,141,748	America

Luego podemos Filtrar datos: por ejemplo (Filtrar por continentes)

```
Seleccione un continente [1-5]: 1
```

Nombre	Población	Superficie	Continente
Argentina	45,376,763	2,780,400	América
Brasil	213,993,437	8,515,767	América

Procedemos a realizar el ordenamiento de países por población en orden descendente:

```
Opción (1-4): 2
¿Ordenar de forma ascendente o descendente?
1 - Asc | 2 - Des: 2
```

Nombre	Población	Superficie	Continente
Brasil	213,993,437	8,515,767	América
Japón	125,800,000	377,975	Asia
Alemania	83,149,300	357,022	Europa
Argentina	45,376,763	2,780,400	América

Mostramos estadísticas de nuestros países cargados:

```
Total de paises: 4

-----
Población
-----
Mayor: Brasil (213,993,437 hab.)
Menor: Argentina (45,376,763 hab.)
Promedio de población: 117,079,875.00hab.
-----
Superficie
-----
Promedio de superficie: 3,007,791.00km.
-----
Países por continentes
-----
America:: 2 paises.
Africa: : 0 paises.
Europa: : 1 paises.
Asia:   : 1 paises.
Oceania:: 0 paises.
```

DIFICULTADES Y CONCLUSIONES:

Decisiones:

- Nos planteamos de qué forma podríamos buscar los países.
- Se decidió hacer una función que cargue los datos a una lista de diccionarios extrayendo los datos del csv para su posterior manipulación.
- Nos organizamos a través de Jira y en llamada para coordinar y trabajar en conjunto, resolviendo problemas que encontrábamos o proponiendo en tiempo real nuevas soluciones.
- Se discute de forma eufórica las distintas formas de validar las opciones, pudiendo llegar a un consenso óptimo y eficiente para el sistema.
- Decidimos en algunos casos agregar funciones para mejorar la experiencia de nuestro sistema al usuario final.
- Nos planteamos al inicio de la maquetación del sistema, que pasaría con los países o continentes que tuvieran acentos, véase su solución más adelante.
- Se decidió agregar la validación de que no ingrese caracteres especiales y al mismo tiempo poder agregar espacios en las palabras como Reino Unido, **solución: se usó función replace**

Dificultades:

- Manipulación de entradas de países o continentes con acentos.
- Dificultad para leer el archivo.
- Duplicidad de datos a querer añadir más datos.
- Falta de costumbre al no guardar los archivos.

Conclusiones:

- Se busca y analiza la librería unicodedata, que normaliza las palabras y permite trabajar con acentos de forma natural (creamos una función que permite normalizar texto)
- Propusimos realizar una validación con la librería os, para encontrar la ruta exacta del archivo y evitar futuros problemas.
- Optimizamos el código mediante la investigación de lambda, para poder tomar el máximo y el mínimo de forma visible para el código más simple, priorizando menos líneas, menos complicación, mayor optimización y entendimiento).

BIBLIOGRAFÍA:

Libro de trabajo Python:

<https://automatetheboringstuff.com/#toc>

Piensa en Python (español):

<https://github.com/espinoza/ThinkPython2-spanish/blob/master/book/thinkpython2-spanish.pdf>
espinoza (Github).

Material de cátedra: Tecnicatura Universitaria en Programación, Programación 1.

Documentos de Python (Librería/Unicodedata):

<https://docs.python.org/es/3.14/library/unicodedata.html#unicodedata.normalize>

Stack Overflow - Preguntas de foro:

<https://es.stackoverflow.com/questions/578320/metodo-replace-para-quitar-acentos>

Stack Overflow - Preguntas de foro:

<https://es.stackoverflow.com/questions/216587/c%C3%B3mo-funcionan-key-y-lambda-en-las-funciones-max-o-min>

LINKS REPOSITORIO Y VIDEO:

Repositorio: https://github.com/ClaudioLegrand/gestion_de_paises.git

Video: <https://youtu.be/9BL8QhnT4yc>